

0.107.3. La fonction `cache()` — ôter un lutin

`#include <cache>` : `cache(numero)` ôte le lutin de l'écran.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `cache(numero)` ôte le lutin de l'écran : il est rangé hors de la vue. Un `sprite()` le ramène.

La ligne à écrire : `#include <cache>`.

Son argument : le numéro du lutin.

Ce qu'elle coûte : environ **95 octets** avec la mise en place des lutins ; l'appel seul, quelques octets.

Essai : fais-le clignoter : `sprite` les images paires, `cache` les impaires.

On retrouve `cache` dans 2 leçons

[0.108](#), [131](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **226 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	47 o	← la console, à l'allumage — la mise en route de la console
main()	44 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
Tuiles	16 o	← CopierTuiles — 1 lettre sur 43
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins

Lettres gravées : A

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

<code>sprite()</code>	× 1	23 o sur place, dans main()
<code>image()</code>	× 1	6 o sur place, dans main() + AttendreImage (28 o)
<code>cache()</code>	× 1	4 o sur place, dans main()
<code>images()</code>	× 1	2 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <cache> : ôter un lutin ----
// Le A est là au départ ; à l'image 120, il part.

#include <sprite>
#include <cache>      // cache() : ôter un lutin
#include <ALPHABET>
```

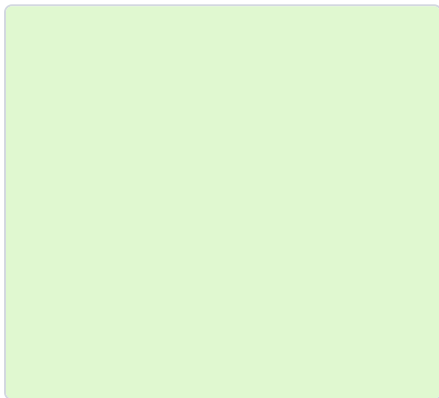
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    sprite(0, 76, 68, ALPHABET[0]);

    while (true) {
        image();
        if (images() == 120) {
            cacher(0);           // le lutin 0 quitte l'écran
        }
    }
}
```

3. Ce qui se passe, étape par étape

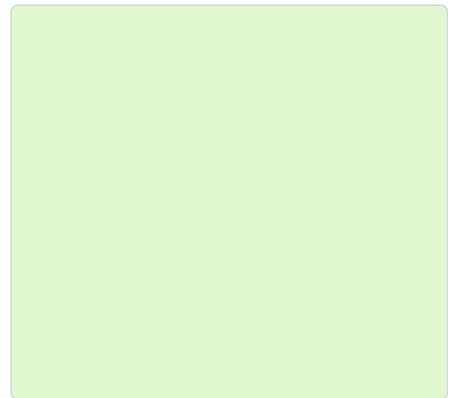
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A au milieu, qui disparaît au bout de deux secondes.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, le A est en (76, 68)
- ✓ puis il est hors de l'écran (76, -16)

Ensuite : **0.107.4. La fonction couleurLutin() — une couleur des personnages** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++